JavaScript Let

Back to JS page

Table of Contents

Block Scope

Before ES6 (2015), JavaScript did not have **Block Scope**.

ES6 introduced the `let` and `const` keywords which provided **Block Scope**.

Variables declared inside a `{ }` block cannot be accessed from outside the block:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Let</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Block Scope</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
{
  let x = 2;
  document.getElementById("demo1").innerHTML = "Inside block: x = " + x;
}
// x can NOT be used here (outside the block)
document.getElementById("demo2").innerHTML = "Outside block: x is not accessible";
</script>

</body>
</html>
```

Example 1

Result View Example

Function Scope

Inside a function all variables declared with `var`, `let` or `const` have **Function Scope**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Let</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Function Scope</h4>
<p id="demo"></p>

<script>
function myFunction() {
  let carName = "Volvo";
  document.getElementById("demo").innerHTML = "Inside function: " + carName;
}
myFunction();
// carName is NOT accessible outside the function
</script>

</body>
</html>
```

Example 2

Result View Example

Global Scope

Variables declared with `var` always have **Global Scope**.

Variables declared with `var` inside a `{ }` block can be accessed from outside the block:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Let</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Global Scope (var)</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
{
  var x = 2;
  document.getElementById("demo1").innerHTML = "Inside block: x = " + x;
}
// x CAN be used here (var has global scope)
document.getElementById("demo2").innerHTML = "Outside block: x = " + x;
</script>

</body>
</html>
```

Example 3

Result View Example

Cannot be Redeclared

Variables defined with `let` **can not** be redeclared.

You can not accidentally redeclare a variable declared with `let`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Let</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Cannot be Redeclared</h4>
<p>let variables cannot be redeclared in the same scope.</p>
<p id="demo"></p>

<script>
let x = "John Doe";
document.getElementById("demo").innerHTML = x;
// let x = 0; // This would cause an error (commented out)
</script>

</body>
</html>
```

Example 4

Result View Example

Redeclaring Variables

Redeclaring a variable using the `var` keyword can impose problems.

Redeclaring a variable inside a block will also redeclare the variable outside the block.

Redeclaring a variable using the `let` keyword solves this problem - redeclaring inside a block will not redeclare the variable outside the block:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Let</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Redeclaring Variables (let)</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
let x = 10;
// Here x is 10
{
  let x = 2;
  // Here x is 2
  document.getElementById("demo1").innerHTML = "Inside block: x = " + x;
}
// Here x is 10 (not affected by block)
document.getElementById("demo2").innerHTML = "Outside block: x = " + x;
</script>

</body>
</html>
```

Example 5

Result View Example

Let Hoisting

Variables defined with `var` are **hoisted** to the top and can be initialized at any time. You can use the variable before it is declared.

Variables defined with `let` are also hoisted to the top of the block, but not initialized. Using a `let` variable before it is declared will result in a `ReferenceError`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Let</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Let Hoisting</h4>
<p id="demo"></p>

<script>
try {
  // Using let variable before declaring it causes ReferenceError
  carName = "Volvo";
  let carName;
} catch(err) {
  document.getElementById("demo").innerHTML = "Error: " + err.message;
}
</script>

</body>
</html>
```

Example 6

Result View Example

Document

Document in project

You can Download PDF file.

Reference

- W3Schools JavaScript Let